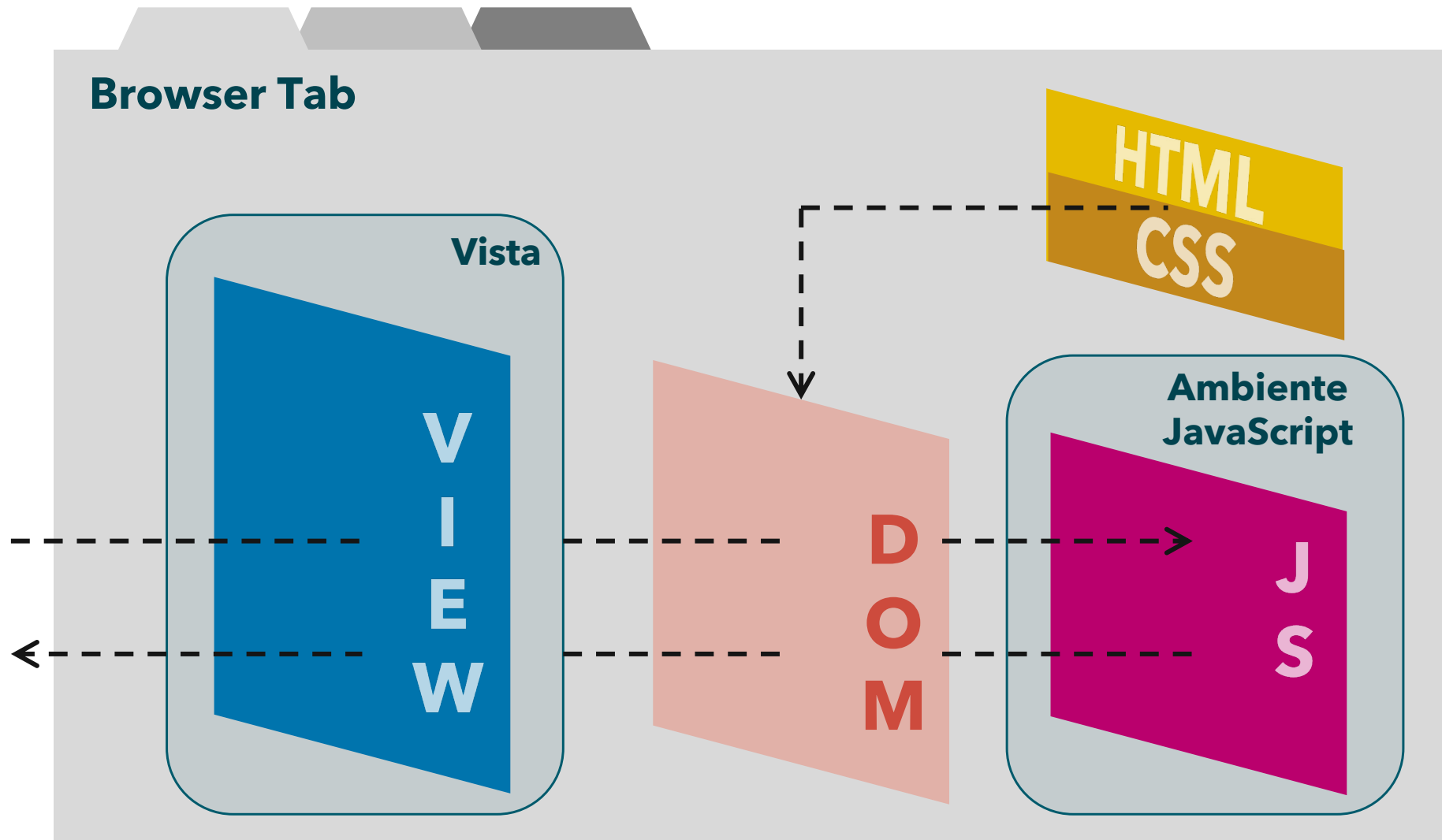


Il browser come ambiente di esecuzione

UI View & UI Logic



...

Il browser utilizza l'HTML e il CSS per inizializzare il **DOM (Document Object Model)** della pagina.

Il Document Object Model

È la struttura dati che mantiene in memoria tutte le informazioni sulla visualizzazione della pagina. Viene **inizializzata** a partire da HTML e CSS ma nel corso della sua vita può essere **modificata**. La vita del DOM termina quando la pagina viene chiusa o ricaricata.

...

Document Object Model

struttura dati che rappresenta il documento HTML

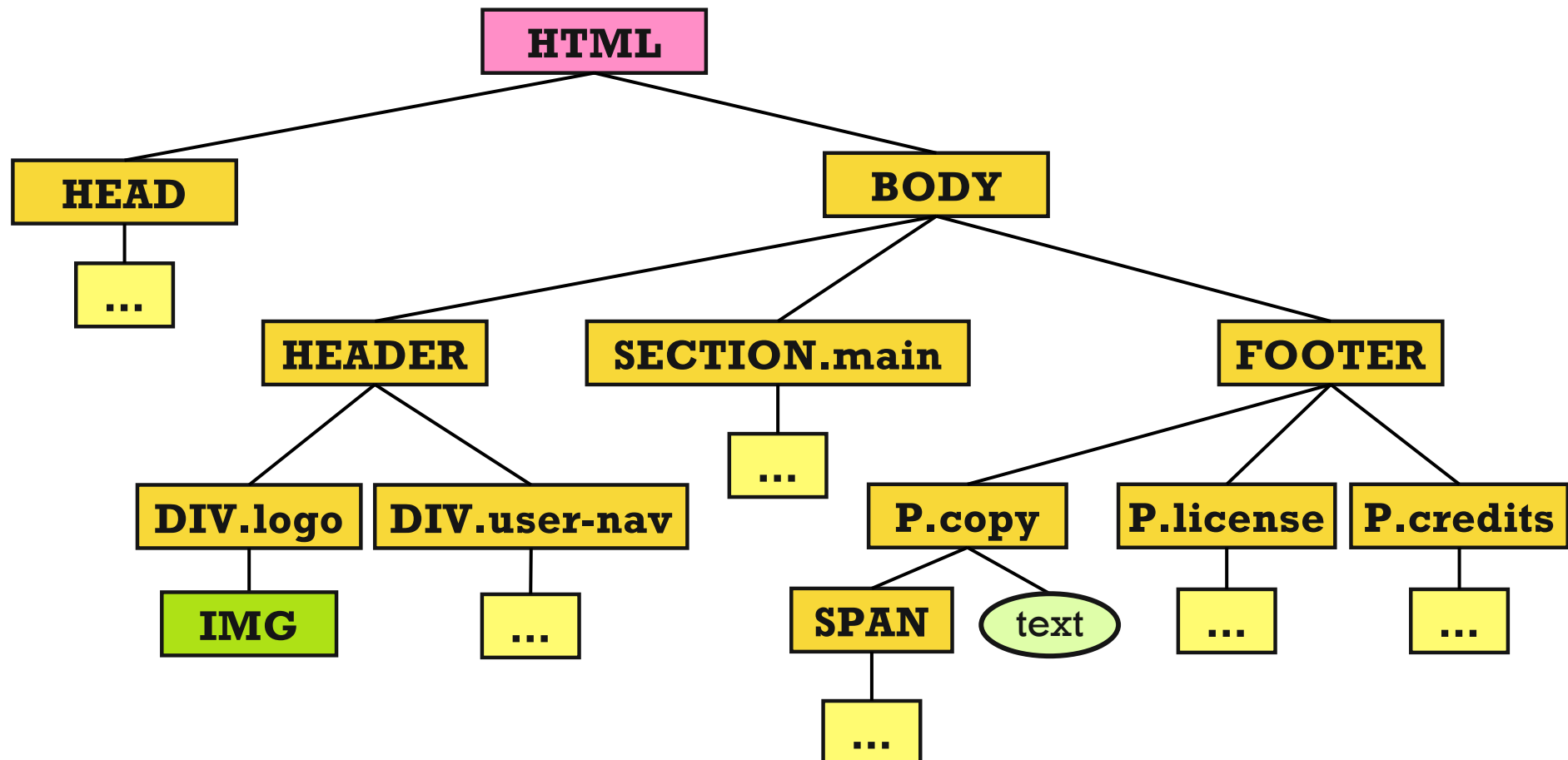
nell'ambiente-browser è disponibile un oggetto **document** che contiene il DOM del documento

```
<html lang="en">
  ▶ <head> ... </head>
  ▼ <body> grid
    ▼ <header> flex
      ▶ <div class="logo"> ... </div>
      ▶ <nav class="user-nav"> ... </nav> flex
    </header>
    ▶ <section class="main"> ... </section> grid
    ▼ <footer> grid
      ▼ <p class="copy">
        <span class="highlight">Lend-A-Book</span>
        " è un progetto no profit realizzato per il corso di Tecnologie Web, CdL in Informatic
      </p>
      ▶ <p class="license"> ... </p>
      ▶ <p class="credits"> ... </p>
    </footer>
  </body>
</html>
```

Document Object Model

struttura dati che rappresenta il documento HTML

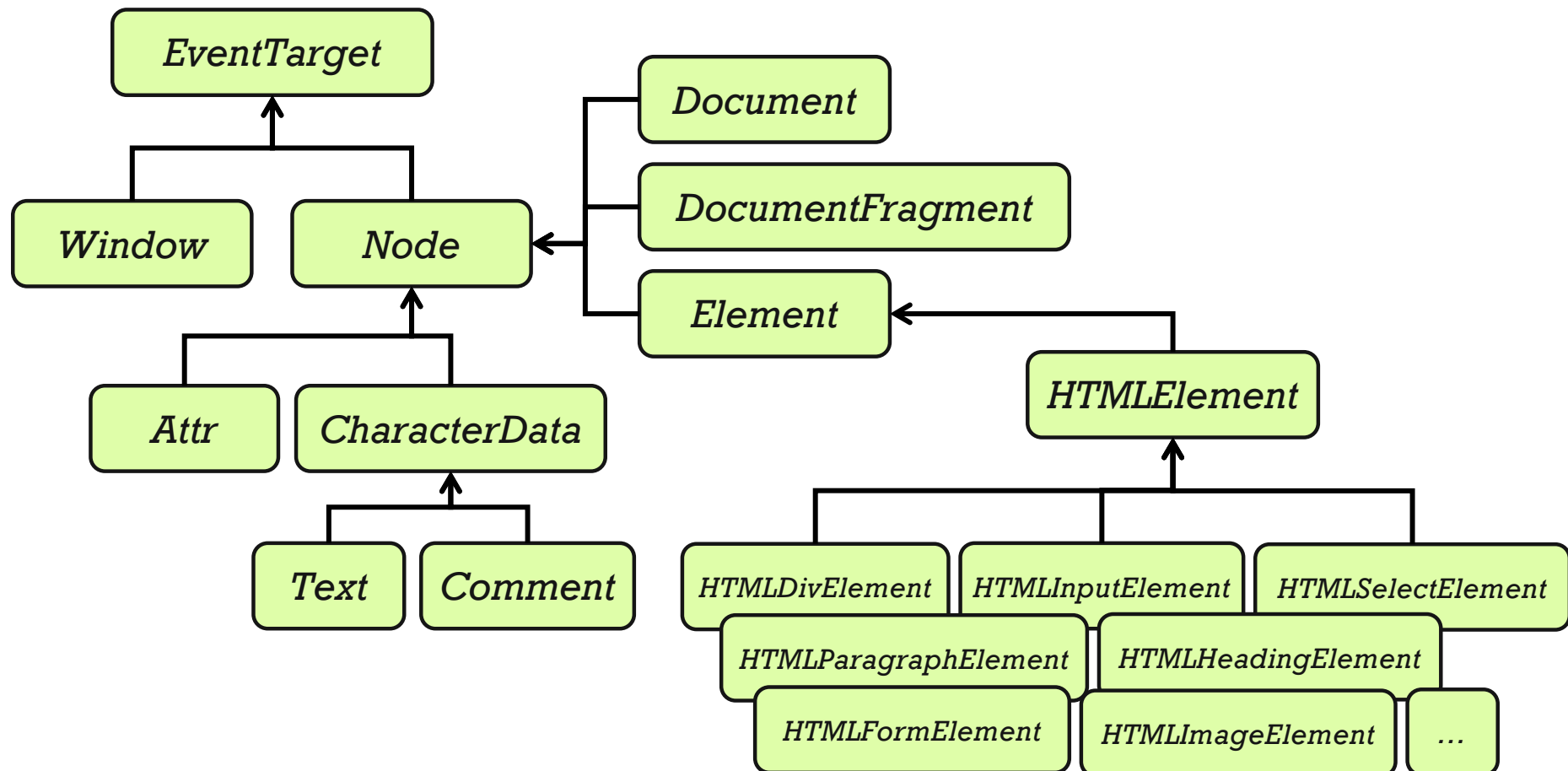
Il **DOM** è un albero: in esso possiamo distinguere il nodo **root**, i nodi **intermedi** (tipicamente elementi html) e le **foglie** (html, testo, commenti...)



Document Object Model

struttura dati che rappresenta il documento HTML

La **DOM API** è costituita da un insieme di **interfacce** e **oggetti** che le implementano



Interfacce

La parola **interfaccia** (*interface*) denota genericamente il «fronte esterno» di un servizio, che permette ad altri di utilizzarlo.

La **UI** (**user interface**) è il «fronte verso gli utenti» di un applicativo.

L'**API** (**application programming interface**) è invece il «fronte verso altri software» o «verso i programmatori» di un software: ci dice come utilizzarne i servizi all'interno del codice.

L'**object interface** di un oggetto è costituita dalle sue **proprietà** e dalle **signature** dei suoi **metodi**; in altre parole il modo in cui l'oggetto può essere «utilizzato» nel codice, che prescinde da come esso implementa internamente tale comportamento.

L'interfaccia in questa terza accezione **esprime un tipo di dato** ed è un'**astrazione** del concetto di **classe**. La classe infatti contiene anche una componente **operazionale**: dice come si costruiscono gli oggetti di quella classe, e come ciascun metodo implementa il comportamento dichiarato. Una stessa **object interface** può essere implementata da classi diverse.

Interfacce come contratti

La specifica di un'interfaccia (indipendentemente dal livello a cui si situa) può essere *a posteriori* (ossia: realizzo il mio servizio software e poi pubblico la sua API, ossia l'elenco delle funzionalità che possono essere richieste e dei tipi di dati richiesti in input e forniti in output) oppure *a priori*, sotto forma di *contratto* o *specifica*: in questo caso l'elenco delle funzionalità è un requisito di servizio, che può essere implementato da molti fornitori diversi. Analogamente, il *mockup* può essere visto come una specifica a priori di una *user interface*.

La DOM API

La DOM API è precisamente questo secondo tipo di **API**: è una specifica (stabilita dal W3C) di come deve essere il servizio di interrogazione e aggiornamento del DOM. Giacché il servizio è offerto (ai programmatori Javascript e ai loro programmi) dai browser, i browser giocano il ruolo dei fornitori e come tali devono attenersi alla specifica.

La specifica della DOM API è fatta essenzialmente di **oggetti** che devono essere forniti (uno su tutti, il DOM stesso, e gli elementi che lo compongono) e di **object interfaces** che quegli stessi oggetti devono rispettare. Essendo «solo» una specifica, non contiene componenti imperative, quindi non definisce classi. Starà a chi la implementa decidere se utilizzare o meno le classi per realizzare gli oggetti in questione. La verità è che a chi **usa** la DOM API di questo non interessa molto.

Object interface in Javascript

Alcuni linguaggi OOP permettono di dichiarare le **object interface** come costrutti del linguaggio stesso. Sono costrutti privi di semantica operativa: un'interfaccia, come abbiamo detto, non **fa** qualcosa, si limita a **dichiararlo**.

Avere le **object interfaces** come costrutti formali serve solo al **controllo statico dei tipi**. Ne consegue che Javascript, che non prevede controllo statico dei tipi, non le prevede come elementi del linguaggio.

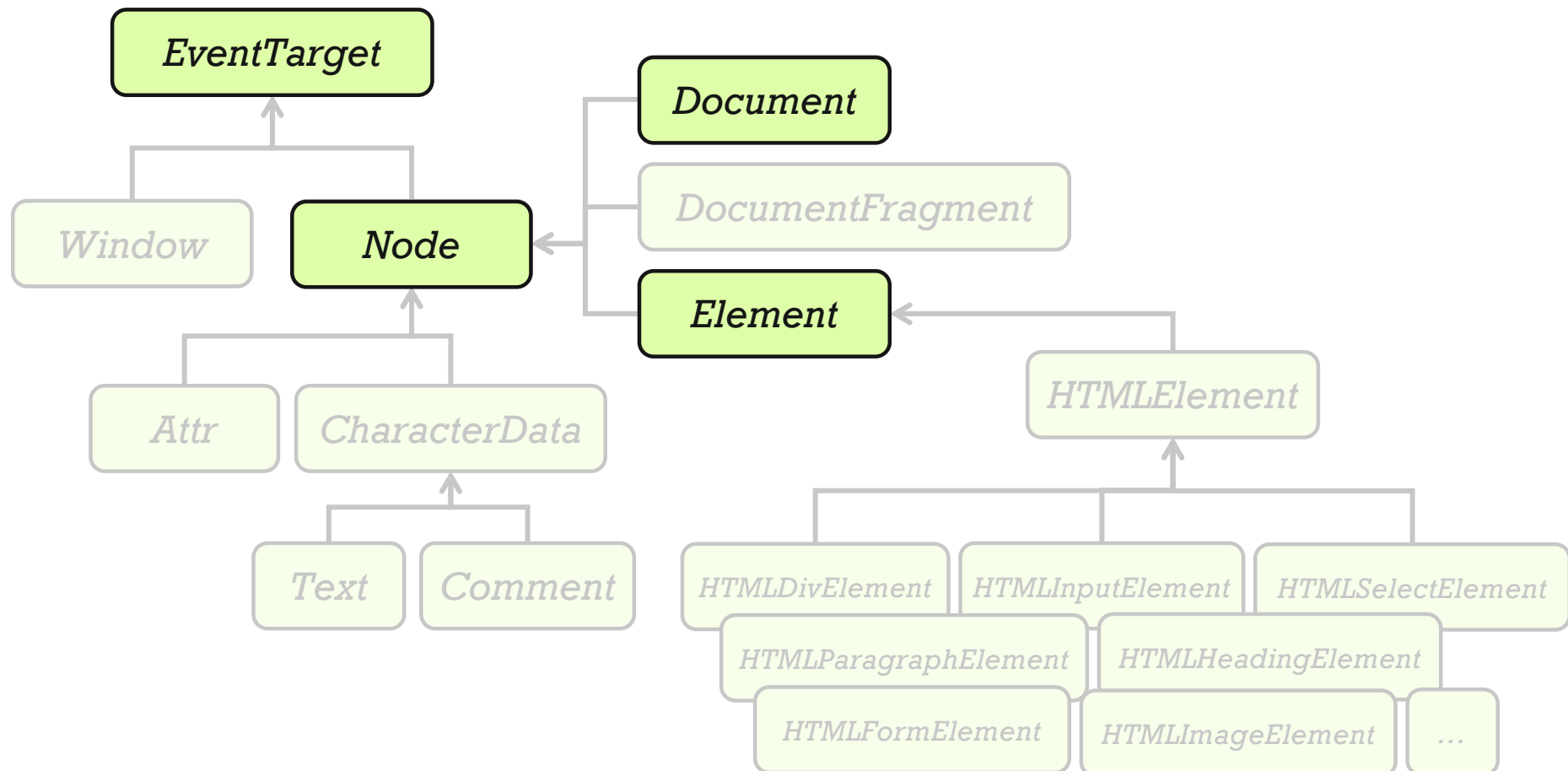
Quando diciamo, ad esempio, che i nodi DIV del **DOM** implementano l'interfaccia *HTMLDivElement*, stiamo semplicemente dicendo che quegli oggetti, indipendentemente da come sono definiti, hanno tutte le proprietà e i metodi elencati nella relativa specifica fornita dal W3C.

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

Document Object Model

struttura dati che rappresenta il documento HTML

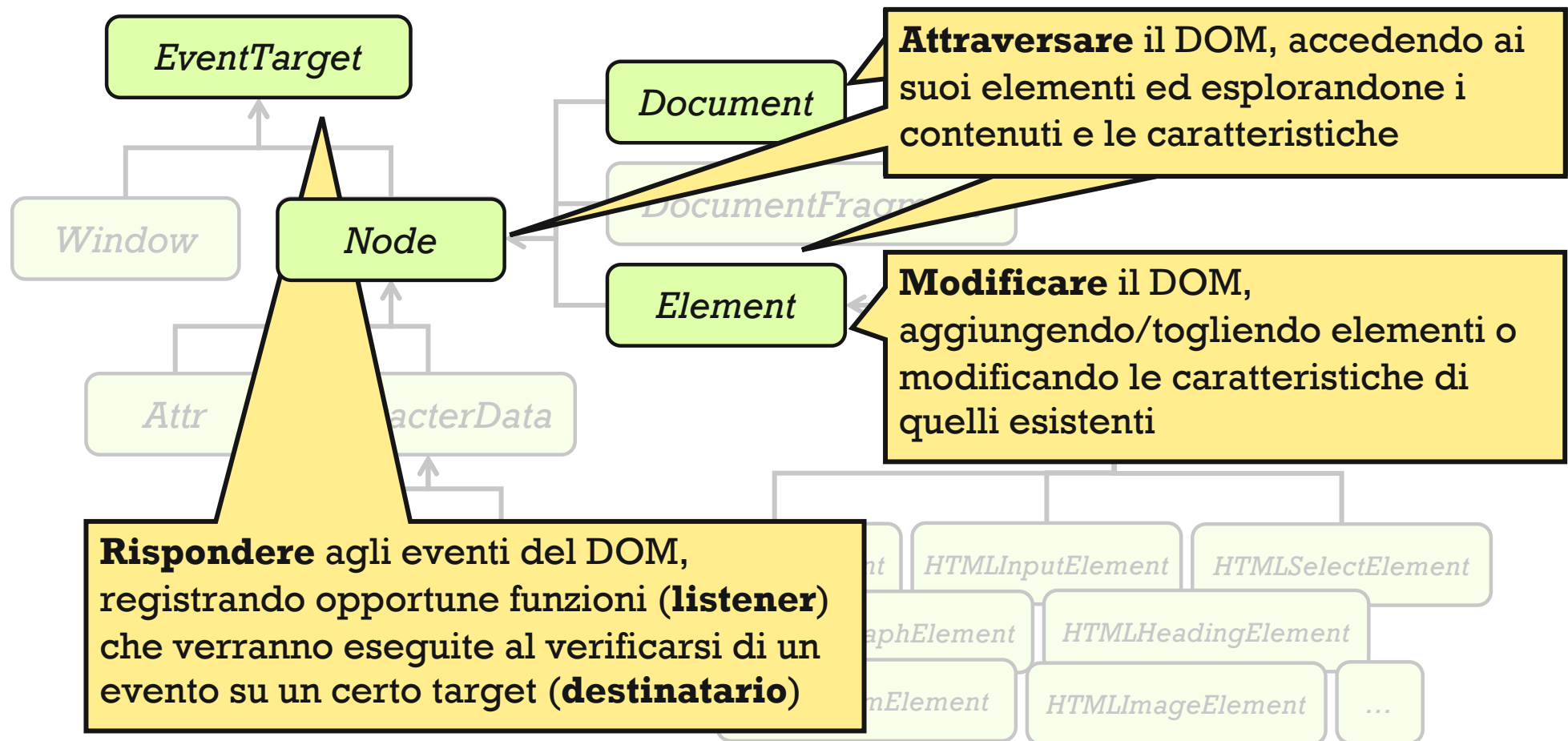
La **DOM API** è costituita da un insieme di **interfacce** e **oggetti** che le implementano



Document Object Model

struttura dati che rappresenta il documento HTML

La **DOM API** è costituita da un insieme di **interfacce** e **oggetti** che le implementano



Interagire con DOM

Le interazioni tra i nostri script e il DOM ricadono in tre macrocategorie:

- **Navigazione o attraversamento del DOM.** In questa categoria rientrano tutte le operazioni volte a **trovare** un elemento o un insieme di elementi del DOM, sulla base della struttura dell'albero e/o del tipo/classe/id dell'elemento stesso, e a **leggerne** le caratteristiche rilevanti. Le operazioni di questa categoria vengono per lo più gestite tramite le proprietà e i metodi dell'interfaccia **Element**.
- **Modifica del DOM.** In questa categoria rientrano le operazioni volte ad aggiungere o rimuovere elementi del DOM, a spostare elementi da un punto all'altro dell'albero, a modificare le caratteristiche (ad es. le classi di appartenenza, o gli attributi in generale). Anche le operazioni di questa categoria vengono per lo più gestite tramite le proprietà e i metodi dell'interfaccia **Element**.
- **Gestione di eventi del DOM.** In questa categoria rientrano le operazioni volte a **registrare (subscribe/unsubscribe)** delle opportune funzioni come «ascoltatori» (**listener**) di degli eventi UI di interesse. La registrazione è specifica per una certa tipologia di evento e per l'elemento a partire dal quale l'evento si origina. I **listener** associati verranno invocati nel momento in cui quello specifico evento si verifica su quello specifico elemento: ciascun **listener** riceverà a quel punto le informazioni dettagliate sull'evento stesso. Le operazioni di questa categoria vengono gestite tramite le proprietà e i metodi dell'interfaccia **EventTarget**

In buona sostanza, alla base della programmazione di una UI in «vanilla Javascript» ci sono variazioni sul seguente loop: (1) trovare nel DOM gli elementi su cui intervenire, (2) registrare su di essi dei listener, (3) quando l'evento si verifica, modificare opportunamente il DOM.

Navigazione del DOM

Come trovare gli elementi che ci interessano

Ricerca per **tipo**, **classe**, **id**

- `getElementById(id) → Element`
- `getElementsByTagName(htmlTag) → HTMLCollection`
- `getElementsByClassName(className) → HTMLCollection`

metodi definiti sia da **Element**
che da **Document**

Attraversamento dell'**albero**

- `document.documentElement → Element`
- `document.head → Element`
- `document.body → Element`
- `element.children → HTMLCollection`
- `node.childNodes → NodeList (L)`
- `node.parentElement → Element`
- `node.parentNode → Node`

proprietà definite da **Document**

proprietà definita da **Element**

proprietà definite da **Node**

Ricerca per **selettore**

- `querySelector(selector) → Element`
- `querySelectorAll(selector) → NodeList (S)`
- `closest(selector) → Element`

metodi definiti sia da **Element**
che da **Document**

metodo definito da **Element**

Consultare, navigare, attraversare il DOM - I

Possiamo cercare uno o più elementi nel DOM a partire dal tag, dalla classe o dall'id. La ricerca può essere effettuata a partire da un dato **element**, nel qual caso avverrà solo nel relativo sottoalbero, oppure a partire dall'oggetto **document** intero.

- **getElementById(id)**: restituisce l'elemento con l'id cercato, se esistente; **null** altrimenti.
- **getElementsByName(htmlTag)**: restituisce una HTMLCollection (live) contenente tutti gli elementi trovati con il tag specificato.
- **getElementsByClassName(className)**: restituisce una HTMLCollection (live) contenente tutti gli elementi trovati che appartengono alla classe specificata.

Partendo dal **document** possiamo risalire alla **radice** dell'albero DOM, così come direttamente all'elemento **head** o **body**:

- **document.documentElement**: la radice dell'albero DOM
- **document.head**: l'elemento **HEAD** del documento
- **document.body**: l'elemento **BODY** del documento

Partendo da qualunque **nodo** dell'albero (che sia o meno un **element**) possiamo trovare i nodi **figli** e il nodo **genitore**:

- **node.childNodes**: una NodeList (live) contenente **tutti i nodi figli** del nodo corrente, che siano **elementi** o meno
- **node.parentElement**: l'elemento genitore, se c'è un genitore e quel genitore è un elemento (e non un altro tipo di nodo); altrimenti **null**
- **node.parentNode**: il nodo genitore, di qualunque tipo esso sia, se c'è un genitore; altrimenti **null**

Partendo da un **elemento** dell'albero possiamo trovare gli **elementi figli**:

- **element.children**: una HTMLCollection (live) contenente tutti i figli dell'elemento attuale che sono a loro volta elementi.

Consultare, navigare, attraversare il DOM - II

Possiamo infine cercare degli elementi utilizzando i **selettori** che abbiamo già visto per i CSS. La ricerca **discendente** può essere fatta sia partendo dall'intero **document**, sia partendo da un qualunque **elemento** dell'albero:

- **querySelector(selector)**: restituisce il **primo elemento** trovato nella ricerca che corrisponde al selettore, se esistente; **null** altrimenti.
- **querySelectorAll(selector)**: restituisce una NodeList (statica) contenente tutti gli elementi trovati che corrispondono al selettore.

La ricerca **ascendente** (verso la radice) può essere fatta solo partendo da un **elemento** dell'albero:

- **closest(selector)**: restituisce il **primo elemento** trovato risalendo verso la radice che corrisponde al selettore, se esistente; **null** altrimenti.

HTMLCollection e NodeList

HTMLCollection e **NodeList** sono due tipi di «collezioni» **array-like**, ma non sono array. Questo significa che non è possibile usare su di esse i metodi predefiniti della classe Array. Tuttavia il fatto che siano **array-like** ci permette di effettuare sia i cicli **for** tradizionali, sia i cicli **for..of**. Introdotte agli albori della DOM Api quando gli array standard di Javascript non avevano caratteristiche adeguate, queste strutture dati sono state mantenute per retrocompatibilità.

Importante: La ricerca per tag, id o classe, e le proprietà che permettono di attraversare l'albero, restituiscono collezioni **live** (ossia, in diretta): se la struttura del DOM cambia, i contenuti della collezione si modificano. Questo sia per le HTMLCollection che per le NodeList.

La ricerca basata su selettore invece restituisce una collezione **statica**: se il contenuto del DOM si modifica sarà necessario ripetere la query per ottenere i risultati aggiornati.

Possiamo trasferire i contenuti di una HTMLCollection o di una NodeList in un array, se ci serve:

```
const myArray = Array.from(nodeListOrCollection);
```

tuttavia dobbiamo sapere che l'array risultante sarà sempre e comunque una collezione statica.

Modificare il DOM

Modificare, inserire ed eliminare elementi

Modifica **bulk**

- `element.innerText → string`
- `element.innerHTML → string`

il contenuto dell'elemento, rispettivamente come rappresentazione testuale o come HTML

modificando questi valori, si sostituisce completamente il sottoalbero che fa capo all'elemento.

Creazione

- `document.createElement(tag)`

crea un **Element** con il tag HTML specificato

l'elemento **non** viene aggiunto nel DOM, solo creato come radice di un altro sottoalbero

Rimozione

- `element.remove()`

elimina l'elemento dal DOM

Aggiunta di nodi all'albero

- `element.insertAdjacentElement(where, anElem)`
- `element.insertAdjacentHTML(where, htmlString)`
- `element.insertAdjacentText(where, textString)`

inserisce un intero sottoalbero in posizione contigua all'elemento di partenza

inserisce un nodo Text in posizione contigua all'elemento di partenza.

L'aggiunta di **text/HTML** comporta la **creazione** di nuovi elementi. L'aggiunta di un **element**, se esso apparteneva già al DOM, ne comporta lo **spostamento**.

Modificare la struttura del DOM

Il modo più «ortodosso» di modificare la struttura del DOM è usare uno dei tre metodi:

- **element.insertAdjacentElement(**where**, anElem)**: **anElem** può essere un elemento precedentemente creato con **document.createElement**, quindi non ancora appartenente al DOM, o un elemento già presente nel DOM. In questo secondo caso esso verrà **spostato** (con tutto il sottoalbero che gli fa capo) alla posizione specificata.
- **element.insertAdjacentHTML(**where**, htmlString)**: la stringa HTML viene usata per costruire un **nuovo** sottoalbero, il cui nodo radice viene inserito alla posizione specificata.
- **element.insertAdjacentText(**where**, textString)**: la stringa di testo viene usata per creare un nuovo nodo **Text**, che viene inserito alla posizione specificata.

La posizione è relativa all'**element** su cui viene chiamato il metodo, ed è espressa da una stringa fra le seguenti:

- **beforebegin** – il nodo cardine (sia esso l'elemento indicato da **anElem**, la radice del sottoalbero creato dalla **htmlString**, o il nodo testuale creato dalla **textString**) viene inserito come **fratello predecessore** di **element** (subito prima del tag di apertura di **element**)
- **afterbegin** – il nodo cardine viene inserito come **primo figlio** di **element** (subito dopo il tag di apertura di **element**)
- **beforeend** – il nodo cardine viene inserito come **ultimo figlio** di **element** (subito prima del tag di chiusura di **element**)
- **afterend** – il nodo cardine viene inserito come **fratello successore** di **element** (subito dopo il tag di chiusura di **element**)

Modificare il DOM

Modificare classi e attributi

Modifica delle **classi**

- `element.classList.add(nomeClasse)`
- `element.classList.remove(nomeClasse)`
- `element.classList.toggle(nomeClasse)`
- `element.classList.has(nomeClasse)`

aggiunge o toglie una classe a quelle elencate nell'attributo **class**

dice se l'elemento possiede una data classe

poiché modificando le classi a cui l'elemento appartiene cambiano le regole di stile che gli si applicano, questo costituisce un modo molto efficace di ottenere effetti di interattività, quali far apparire/scompare elementi, evidenziarli con colori o caratteri diversi, spostarli di posizione sullo schermo, e così via.

Modifica dei valori degli **attributi**

- `element.getAttribute(attrName)`
- `element.setAttribute(attrName, value)`

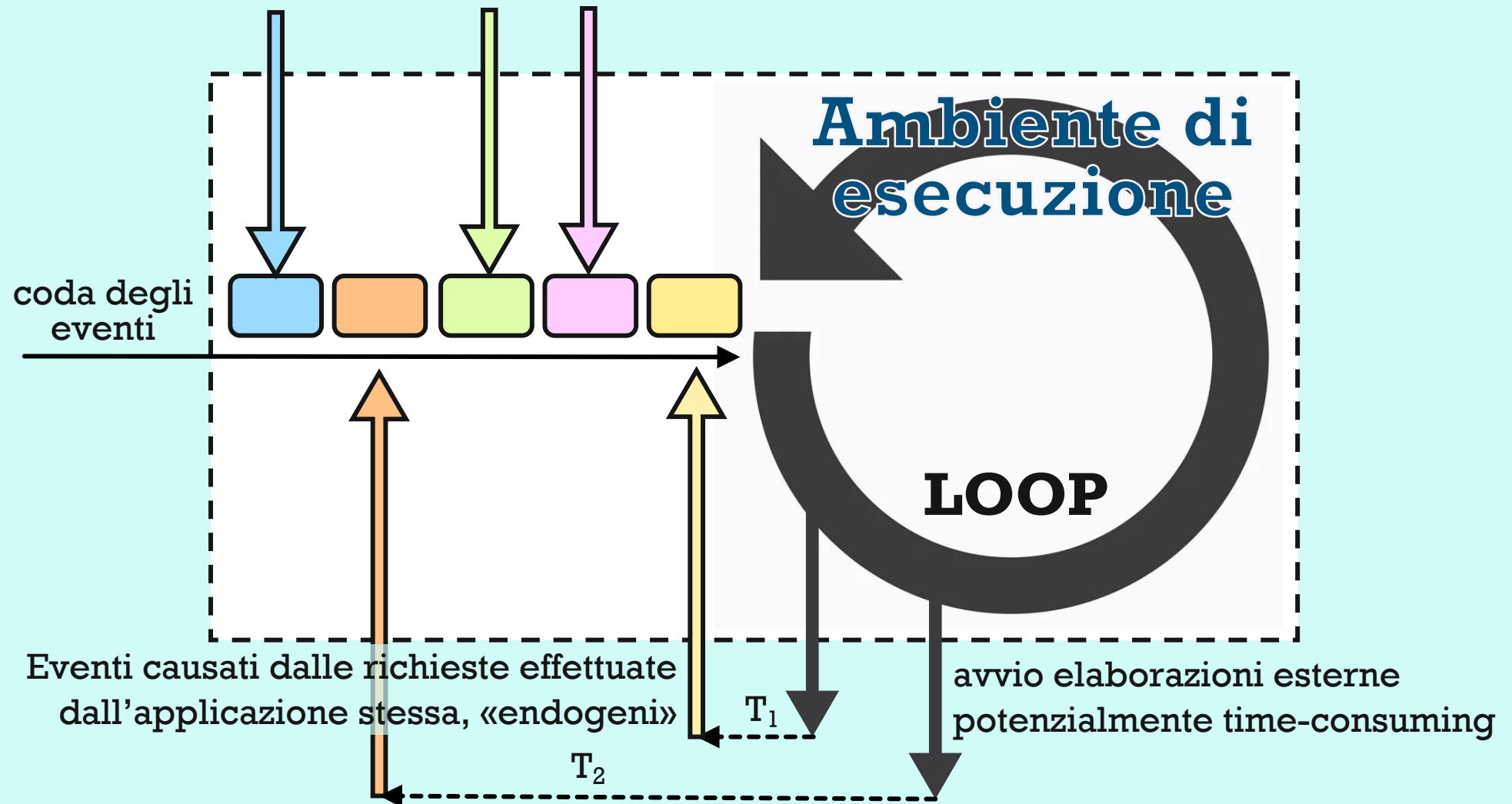
restituisce il valore dell'attributo specificato sotto forma di stringa

assegna il valore **value** all'attributo specificato

Javascript e gli eventi

asynchronous single-thread event loop

Eventi «spontanei» generati all'esterno, «esogeni»



Asynchronous single-thread event loop

L'ambiente di esecuzione di Javascript interagisce con il mondo esterno tramite **eventi**. Gli eventi giungono dall'esterno e possono essere:

- **esogeni** ossia generati spontaneamente da qualche agente esterno all'applicazione (ad esempio, l'utente); come tali completamente imprevedibili: non è dato dire se e quando avverranno.
- **endogeni** ossia conseguenza di una richiesta al mondo esterno generata all'interno dell'applicazione; in questo caso è possibile prevedere con ragionevole certezza che si verificheranno, a meno di errori nell'elaborazione della richiesta stessa.

Gli eventi si mettono in coda e, perché vengano gestiti, è necessario che all'ambiente di esecuzione sia nota la funzione che risponde all'evento stesso.

Queste funzione è detta

- **listener** nel caso degli eventi esogeni (sta in **ascolto** in attesa del verificarsi dell'evento, che non è prevedibile, e per ogni volta che l'evento si verifica, essa risponde)
- **callback** nel caso di eventi endogeni: tipicamente ad *ogni* avvio della richiesta al mondo esterno, viene specificata la funzione da **richiamare** (call back) quando la richiesta è completata

La logica che c'è dietro però è la medesima: ci sono istruzioni espressamente volte ad associare ad un determinato tipo di evento una funzione, che verrà eseguita quando l'evento si verifica.

Javascript «consuma» la coda degli eventi con un **singolo thread**^(*), dunque gli eventi vengono gestiti nell'ordine in cui si presentano, in sequenza.

Ne consegue che, se la gestione di un evento intraprende un'elaborazione lunga, essa blocca interamente l'applicazione. Per questa ragione le elaborazioni time-consuming (ad esempio, le richieste HTTP, o la scrittura di dati nel DB interno al browser) vengono gestite in modo asincrono. Le istruzioni che avviano tali elaborazioni non si bloccano in attesa del risultato, ma chiedono al programmatore di specificare una funzione (la **callback**, appunto) che verrà invocata quando il calcolo sarà terminato.

(*) è in realtà possibile eseguire alcuni compiti specifici, che non interagiscono con il DOM, in background, in thread paralleli chiamati Web Workers; ciò non toglie che il **core** dell'esecuzione di un programma Javascript abbia le caratteristiche qui descritte.

Gli eventi del DOM

Interagire con la UI

Gli oggetti del DOM ricevono **eventi** in corrispondenza delle interazioni con l'utente da essi supportate

Tutti gli **elementi** supportano interazioni tramite dispositivi di puntamento (mouse, trackpad); nei browser per dispositivi con touch screen è possibile trattare separatamente gli eventi touch o assimilarli a quelli del mouse

click	mouseenter	mouseleave	mouseout
mousedown	mouseup	mouseover	mousemove

Molti elementi di **input** supportano l'interazione con la tastiera

focus	blur	keydown	keyup
-------	------	---------	-------

Gli elementi **form** supportano interazioni di livello più alto, corrispondenti ai due tipi di azione possibili:

submit	reset
--------	-------

L'oggetto **document** supporta l'evento
scroll

L'oggetto **window** supporta l'evento
resize

Questi sono solo alcuni degli eventi possibili!
L'elenco esaustivo è lunghissimo.

D'altra parte, va detto che per la maggior parte delle applicazioni gli unici eventi che necessitano di una gestione specifica sono quelli del mouse...

Gli eventi del DOM

Registrare un **listener** per un evento

Tutti gli oggetti del DOM che ricevono eventi implementano il metodo **addEventListener**

```
domObj.addEventListener(nomeEvento, listener);
```

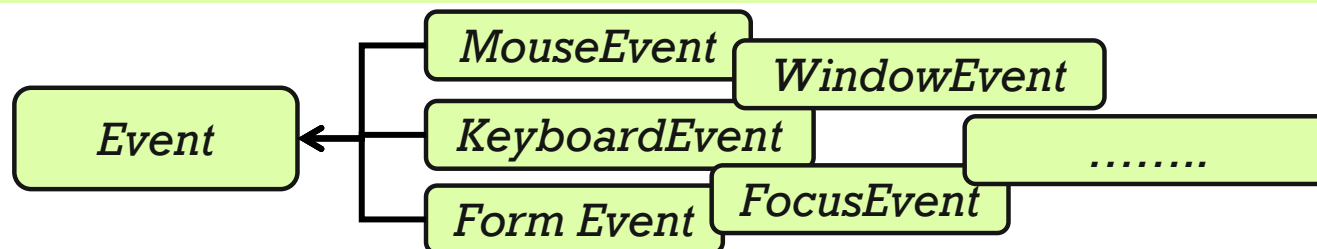
- **nomeEvento** è una stringa che denota la tipologia di evento, ad es. "click", "mouseup", "keydown",...
- **listener** è una funzione che riceve un parametro (opzionale) e non restituisce nulla

La funzione *listener* verrà chiamata quando l'evento **nomeEvento** si verifica sull'oggetto **domObj**; riceverà come parametro un oggetto che contiene le informazioni sull'evento che si è verificato

```
element.removeEventListener(nomeEvento, listener);
```

Rimuove il *listener* dall'elenco degli ascoltatori registrati presso **domObj** per ricevere l'evento **nomeEvento**.

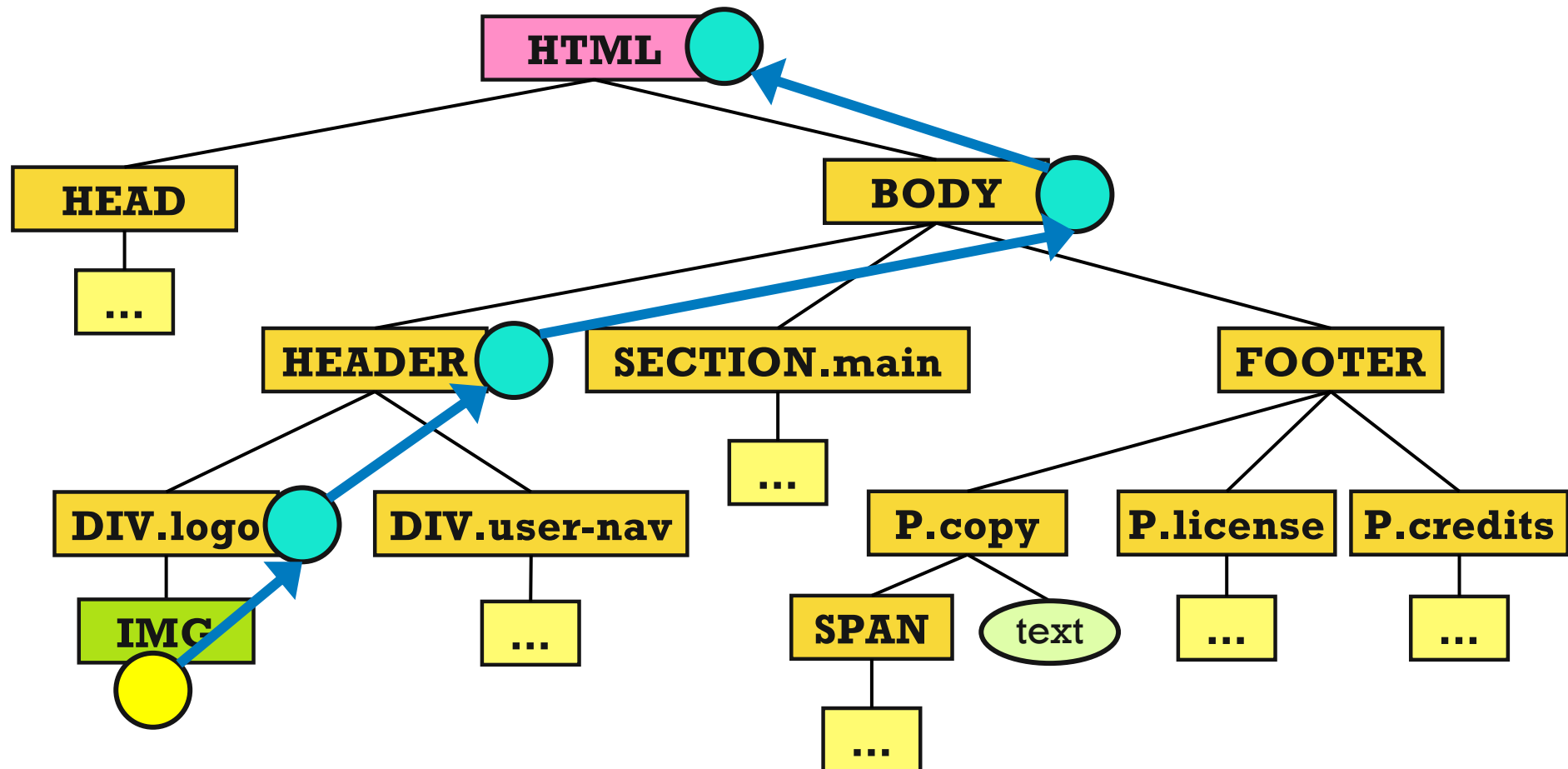
La DOM API definisce **object interfaces** per gli oggetti che rappresentano gli eventi, che vengono ricevuti dai *listener* al verificarsi degli eventi stessi.



Bubbling e capturing

La propagazione degli eventi

Gli eventi **risalgono** l'albero del DOM: questo fenomeno si chiama **bubbling**

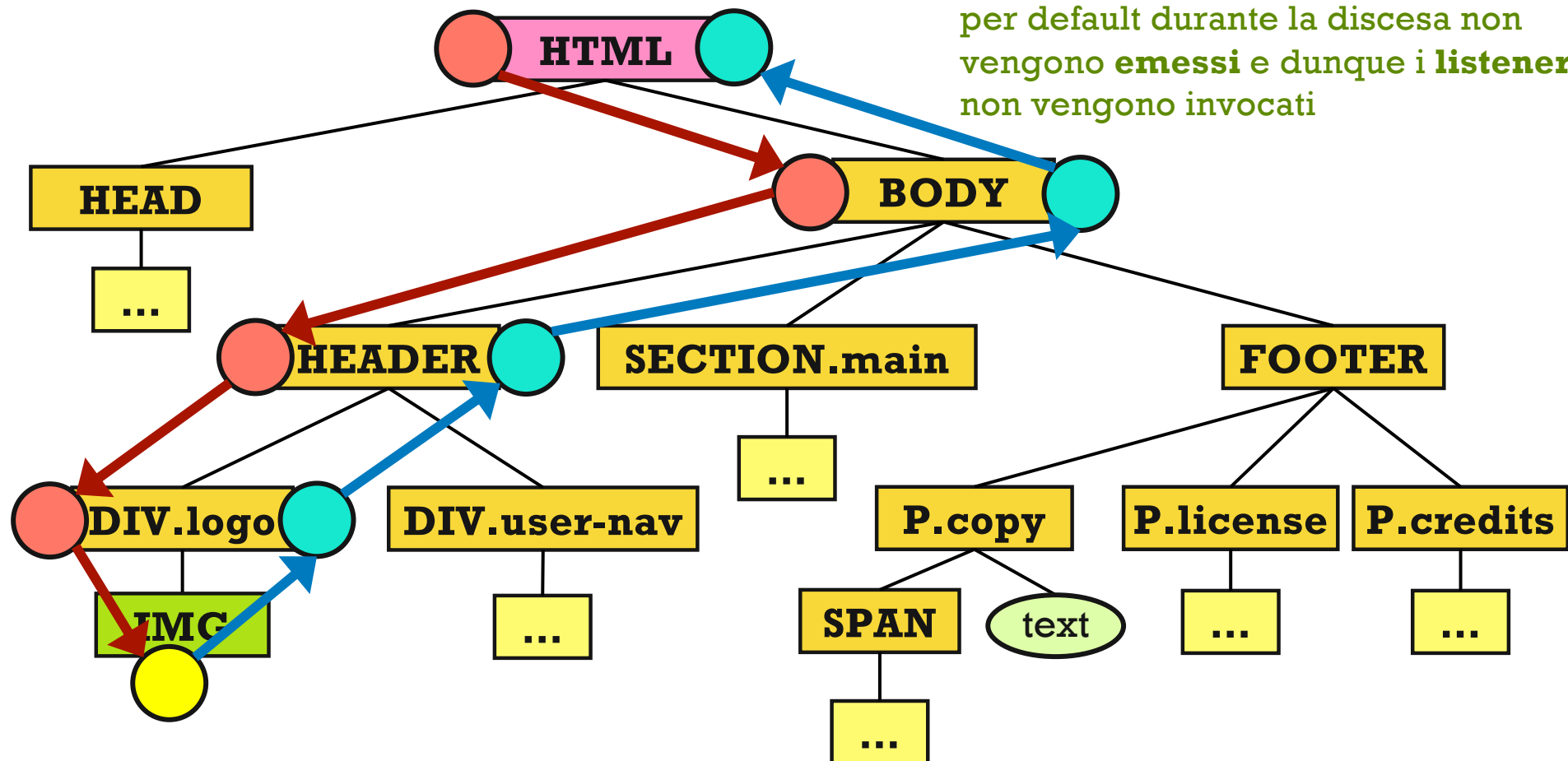


Bubbling e capturing

La propagazione degli eventi

In realtà però gli eventi arrivano all'elemento target **discendendo** l'albero

solo che non ce ne accorgiamo, perché per default durante la discesa non vengono **emessi** e dunque i **listener** non vengono invocati



Bubbling degli eventi

Gli eventi del DOM si generano su un ben preciso elemento.

A partire da quell'elemento vengono propagati sino alla radice. Quindi l'evento verrà **emesso** sia dall'elemento sorgente (nell'esempio della slide precedente, IMG) sia da tutti gli elementi sul percorso che risale dall'elemento sorgente alla radice dell'albero (quindi, nell'esempio della slide precedente, DIV.logo, HEADER, BODY e HTML).

All'interno di un **listener** riceviamo come parametro in ingresso l'oggetto **evento**. Da esso possiamo determinare:

- l'elemento sorgente dell'evento: **evento.target**
- l'elemento che lo sta emettendo in quel momento, ossia, presumibilmente, quello a cui abbiamo associato il listener: **evento.currentTarget**

Perché il bubbling?

Perché questo? Proviamo a immaginare di voler sapere quando l'utente clicca sul footer della pagina. Se non ci fosse il bubbling, noi riceveremmo l'evento **click** solo se l'utente clicca su un punto del footer dove non ci sono altri elementi interni, cioè sullo sfondo.

Difficilmente questo è ciò che ci serve. In generale, quando diciamo «click sul footer» intendiamo «click sul footer **incluso** ciò che esso contiene». Il meccanismo del **bubbling** mette in pratica questo concetto.

Bloccare il bubbling

A volte (e lo vedremo nell'esempio del **glasspane** per le finestre modali) **non** vogliamo che un dato evento si propaghi oltre un dato elemento **el**. In tal caso dovremo **ascoltare** il verificarsi dell'evento su **el**, e all'interno del listener usare l'istruzione:

`evento.stopPropagation();`

Capturing degli eventi

La specifica del DOM stabilisce che l'evento **giunga** all'elemento target attraversando l'albero in discesa. Questa fase è detta **capturing** e precede il bubbling. Per lo più la fase di capturing è **nascosta**; in questa fase il «passaggio» di un evento da un elemento non provoca di default l'invocazione dei **listener**.

Possiamo però chiedere che un *listener* venga invocato durante il capturing specificando un'apposita opzione del metodo **addEventListener**:

```
domObj.addEventListener(nomeEvento, listener, {capture: true});
```

Anche in questo caso potremo, all'interno del *listener*, bloccare la propagazione con

```
evento.stopPropagation();
```

Bloccando la propagazione in fase di capturing, impediamo che l'evento arrivi al suo reale target e la fase di bubbling non avrà mai inizio. Di solito questo serve in applicazioni che richiedono una gestione totalmente customizzata di certi tipi di eventi.

Effetti di default

Alcuni eventi hanno degli effetti di default: ad esempio il click su un elemento A provoca una navigazione ad altra pagina, il tasto destro fa comparire il menu contestuale, il tasto TAB fa passare da un controllo di input al successivo...

È possibile impedire il verificarsi degli effetti di default all'interno di un *listener* con:

```
evento.preventDefault();
```

E se vogliamo bloccare gli effetti di default in tutta un'area dell'applicazione, conviene farlo in fase di capturing.